Lab Sheet: File Input/Output in Java

Course: OOP Lab / CSE 1116
Topic: Console Input, File Reading & Writing using BufferedReader, Scanner, PrintWriter, BufferedWriter
Level: Beginner to Intermediate
Duration: 3 hours

Prepared by: Ashraful Islam Paran, Dept. of CSE, UIU

1. Objectives

By the end of this lab, students will be able to:

- Understand different ways to read input from console and files in Java.
- Use `BufferedReader` and `InputStreamReader` for efficient console input.
- Use `Scanner` class for flexible input handling from console and files.
- Write data to files using `PrintWriter`.
- Perform file copying using `BufferedReader` and `BufferedWriter`.
- Handle exceptions related to File I/O properly.
- Understand the difference between various I/O approaches.

2. Introduction to File I/O

File I/O (Input/Output) is essential for reading data from external sources and writing results persistently. Java provides several classes in the `java.io` package for handling I/O operations.

Main Approaches Covered:

- Reading from Console using `BufferedReader`
- Reading from Console and Files using `Scanner`
- Writing to Files using `PrintWriter`
- Efficient File Copy using `BufferedReader` + `BufferedWriter`

3. Reading Console Input using BufferedReader

`BufferedReader` wraps around `System.in` for efficient character-based input.

```
import java.io.*;

public class ConsoleReader {
    public static void main(String[] args) {
        try {
            BufferedReader br = new BufferedReader(
                new InputStreamReader(System.in));
```

```
            System.out.print("Enter your name: ");
            String name = br.readLine();
            System.out.println("Hello, " + name + "!");

            br.close();
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

4. Reading Multiple Lines using BufferedReader

```
import java.io.*;

public class MultiLineReader {
    public static void main(String[] args) {
        try {
            BufferedReader br = new BufferedReader(
                new InputStreamReader(System.in));
            String s;
            System.out.println("Enter text (type 'end' to stop):");

            while ((s = br.readLine()) != null && !s.equalsIgnoreCase("end"
                ↪ )) {
                System.out.println("You entered: " + s);
            }
            br.close();
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

5. Using Scanner for Console Input

`Scanner` is more convenient for parsing primitive types.

```
import java.util.Scanner;

public class ScannerDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter an integer: ");
        int num = sc.nextInt();
        System.out.println("You entered: " + num);

        System.out.print("Enter a string: ");
        String str = sc.next();
        System.out.println("You entered: " + str);

        sc.close();
    }
}
```

6. Reading from File using Scanner

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class FileScanner {
    public static void main(String[] args) {
        try {
            File file = new File("input.txt");  // Change path as needed
            Scanner sc = new Scanner(file);

            while (sc.hasNext()) {
                String data = sc.next();
                System.out.println(data);
            }
            sc.close();
        } catch (FileNotFoundException e) {
            System.out.println("File not found: " + e.getMessage());
        }
    }
}
```

7. Writing to File using PrintWriter

```
import java.io.*;

public class FileWriterDemo {
    public static void main(String[] args) {
        try {
            PrintWriter out = new PrintWriter("output.txt");
```

```
            out.println(100);
            out.println("Hello World");
            out.println(45.67);

            out.close();
            System.out.println("Data written successfully!");
        } catch (FileNotFoundException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

8. File Copy using BufferedReader and BufferedWriter

```
import java.io.*;

public class FileCopy {
    public static void main(String[] args) {
        try {
            BufferedReader br = new BufferedReader(new FileReader("a.txt"))
                ↪ ;
            BufferedWriter bw = new BufferedWriter(new FileWriter("b.txt"))
                ↪ ;

            String line;
            while ((line = br.readLine()) != null) {
                bw.write(line);
                bw.newLine();
            }

            br.close();
            bw.close();
            System.out.println("File copied successfully!");
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Tasks

Task 1 – Console Input Processor (20 min)

a. Create a program that reads student information (name, id, cgpa) using `Scanner`.

b. Validate input and display formatted output.

Task 2 – File Reader & Analyzer (25 min)

a. Read numbers from a text file "numbers.txt" using `Scanner`.

b. Calculate and display sum, average, maximum, and minimum.

Task 3 – Student Database Writer (20 min)

a. Create a program that takes student data from console.

b. Write it to a file "students.txt" using `PrintWriter` in a formatted way.

c. Allow multiple entries until user types "exit".

Task 4 – File Merger (25 min)

a. Create two files: "file1.txt" and "file2.txt".

b. Write some content to them.

c. Merge their contents into "merged.txt" using `BufferedReader/BufferedWriter`.

Task 5 – Bank Transaction Correction (25 min)

Due to a software error, every transaction was overcharged by 100 BDT.

a. Read all transactions from `transactions.txt`.

b. Subtract 100 from each transaction amount.

c. Write the corrected transactions to `transactions_updated.txt`.

Format: `SenderName sends ReceiverName : Amount`
Example:

```
Input : Habib sends Labib : 1200
Output: Habib sends Labib : 1100
```

—

Task 6 – Student File Splitter (25 min)

Write a program that reads `student.txt` where each line contains: `ID Name mark1 mark2` (separated by spaces).

Create two files: - `info.txt` → `ID Name` - `mark.txt` → `Name total_marks`
Example:

```
Input : 701 Rabbi 15 12
Output:
  info.txt  → 701 Rabbi
  mark.txt  → Rabbi 27
```

—

Task 7 – Word Counter (20 min)

Read `input.txt` (multiple lines with words separated by spaces).

Count the total number of words (only alphabetic characters) in the entire file. Write the result to `output.txt` in this format:

`Number of words: X` **Keep Coding!** Mastering File I/O is crucial for building real-world Java applications that interact with persistent data.